1)

```
#include <stdio.h>
#include <string.h>

#define MAX 100

struct Date
{
   int day, month, year;
};

struct BloodPressure
{
   int systolic;
   int diastolic;
};

struct Patient
{
   char name[50];
   int age;
   char phone[11];

   struct BloodPressure bp;
   struct Date visitDate;
};

int count = 0;
```

```
void addPatient(struct Patient *p, char n[],
                int age, char phone[],
                int sys, int dia,
                int d, int m, int y)
{
   if (count >= MAX)
   {
      printf("Patient limit reached!\n");
      return;
   }

   strcpy(p[count].name, n);
   p[count].age = age;
   strcpy(p[count].phone, phone);

   p[count].bp.systolic = sys;
   p[count].bp.diastolic = dia;

   p[count].visitDate.day = d;
   p[count].visitDate.month = m;
   p[count].visitDate.year = y;

   count++;
}

void displayPatients(struct Patient p[])
{
```

```
    printf("\nPatient Records:\n");

    for (int i = 0; i < count; i++)
    {
        printf("\nName: %s\n", p[i].name);
        printf("Age: %d\n", p[i].age);
        printf("Phone: %s\n", p[i].phone);

        printf("Blood Pressure: %d/%d\n",
               p[i].bp.systolic,
               p[i].bp.diastolic);

        printf("Visit Date: %02d/%02d/%04d\n",
               p[i].visitDate.day,
               p[i].visitDate.month,
               p[i].visitDate.year);
    }
}

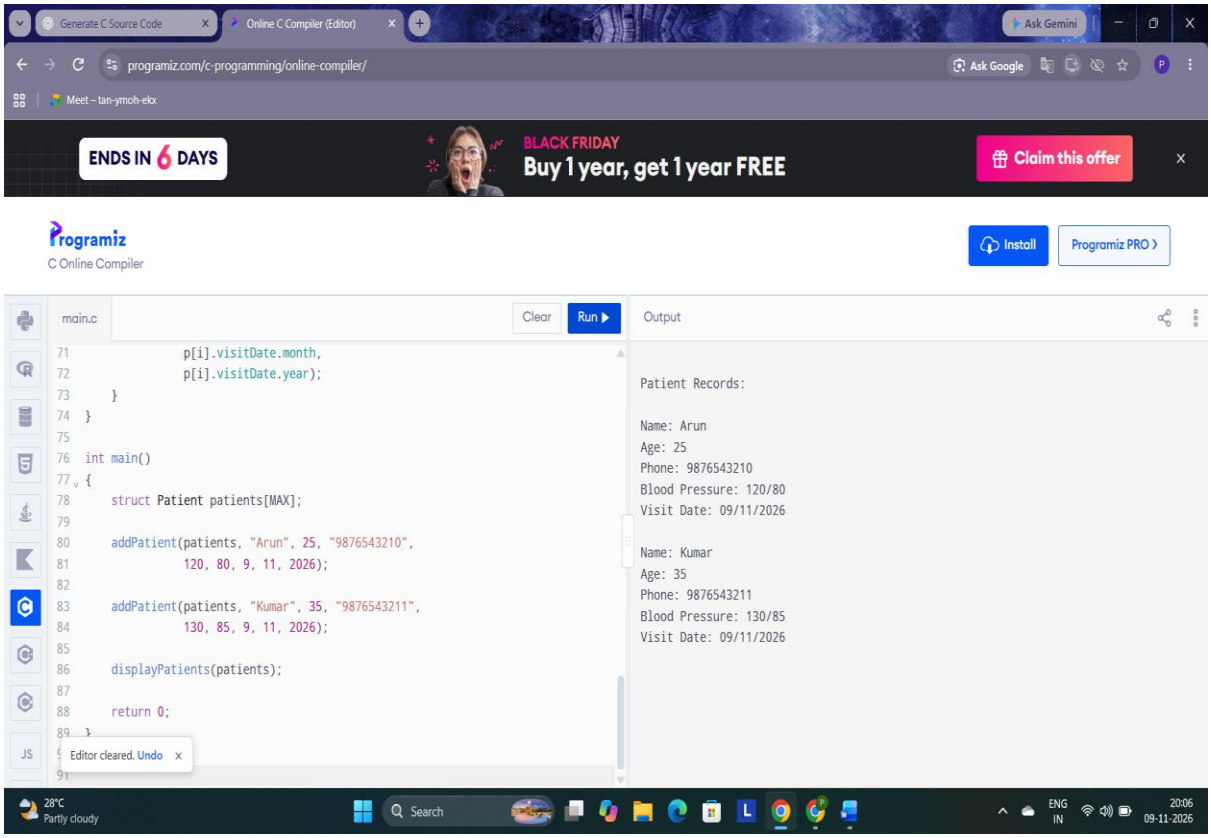
int main()
{
    struct Patient patients[MAX];

    addPatient(patients, "Arun", 25, "9876543210",
               120, 80, 9, 11, 2026);

    addPatient(patients, "Kumar", 35, "9876543211",
               130, 85, 9, 11, 2026);
```

```
    displayPatients(patients);

    return 0;
}
```



2)

```
#include <stdio.h>
#include <string.h>

typedef struct
{
    int day, month, year;
} Date;
```

```
typedef struct
{
   char name[30];
   char city[20];
   int pincode;
} Person;

typedef struct
{
   int id;
   float weight;
   Date booked;
   Person sender;
   Person receiver;
   char service[15];
   float charge;
} Parcel;

void print_parcel(const Parcel *p)
{
   printf("%d %.1f kg %s-%d %02d/%02d/%04d\n",
          p->id,
          p->weight,
          p->receiver.city,
          p->receiver.pincode,
          p->booked.day,
          p->booked.month,
          p->booked.year);
```

```
}

void sort_by_weight(Parcel p[], int n)
{
   for (int i = 0; i < n - 1; i++)
   {
      int min = i;

      for (int j = i + 1; j < n; j++)
      {
         if (p[j].weight < p[min].weight)
            min = j;
      }

      if (min != i)
      {
         Parcel temp = p[i];
         p[i] = p[min];
         p[min] = temp;
      }
   }
}

void search_city(Parcel p[], int n, char city[])
{
   printf("\nParcels for %s:\n", city);

   for (int i = 0; i < n; i++)
```

```
    {
        if (strcmp(p[i].receiver.city, city) == 0)
            print_parcel(&p[i]);
    }
}

int main()
{
    Parcel list[3] =
    {
        {
            501, 4.5,
            {12, 10, 2026},
            {"Arun", "Madurai", 625001},
            {"Kumar", "Madurai", 625001},
            "Normal", 100
        },

        {
            502, 1.2,
            {12, 10, 2026},
            {"Ravi", "Chennai", 600001},
            {"Raja", "Chennai", 600077},
            "Express", 150
        },

        {
            503, 2.8,
```

```
        {13, 10, 2026},
        {"Mani", "Salem", 636001},
        {"John", "Coimbatore", 641001},
        "Normal", 120
    }
};

int n = 3;

sort_by_weight(list, n);

printf("Parcels sorted by weight:\n");

for (int i = 0; i < n; i++)
    print_parcel(&list[i]);

search_city(list, n, "Chennai");

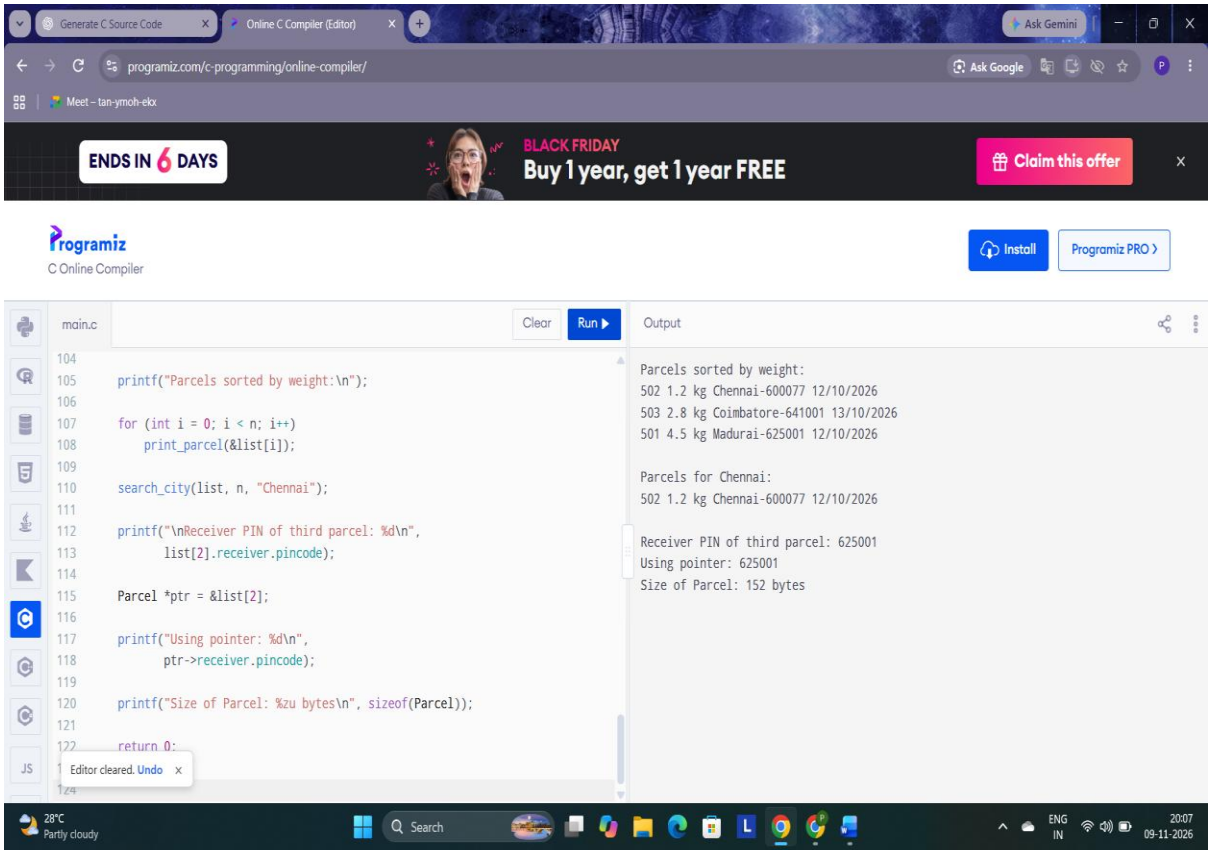
printf("\nReceiver PIN of third parcel: %d\n",
       list[2].receiver.pincode);

Parcel *ptr = &list[2];

printf("Using pointer: %d\n",
       ptr->receiver.pincode);

printf("Size of Parcel: %zu bytes\n", sizeof(Parcel));
```

```
    return 0;
}
```



3)

```
#include <stdio.h>

enum kind
{
   TEMPERATURE,
   HUMIDITY,
   STATUS
};

struct reading
{
```

```
    enum kind type;

    union
    {
        float celsius;
        int percent;
        char status[8];

    } value;
};

void show(const struct reading *r)
{
    switch (r->type)
    {
        case TEMPERATURE:
            printf("Temp %.1f C\n",
                r->value.celsius);
            break;

        case HUMIDITY:
            printf("Humidity %d %%\n",
                r->value.percent);
            break;

        case STATUS:
            printf("Status %s\n",
                r->value.status);
```

```
            break;

        default:
            printf("Invalid reading\n");
    }
}

int main()
{
    struct reading log[3] =
    {
        {TEMPERATURE, {.celsius = 31.5f}},
        {HUMIDITY, {.percent = 68}},
        {STATUS, {.status = "OK"}}
    };

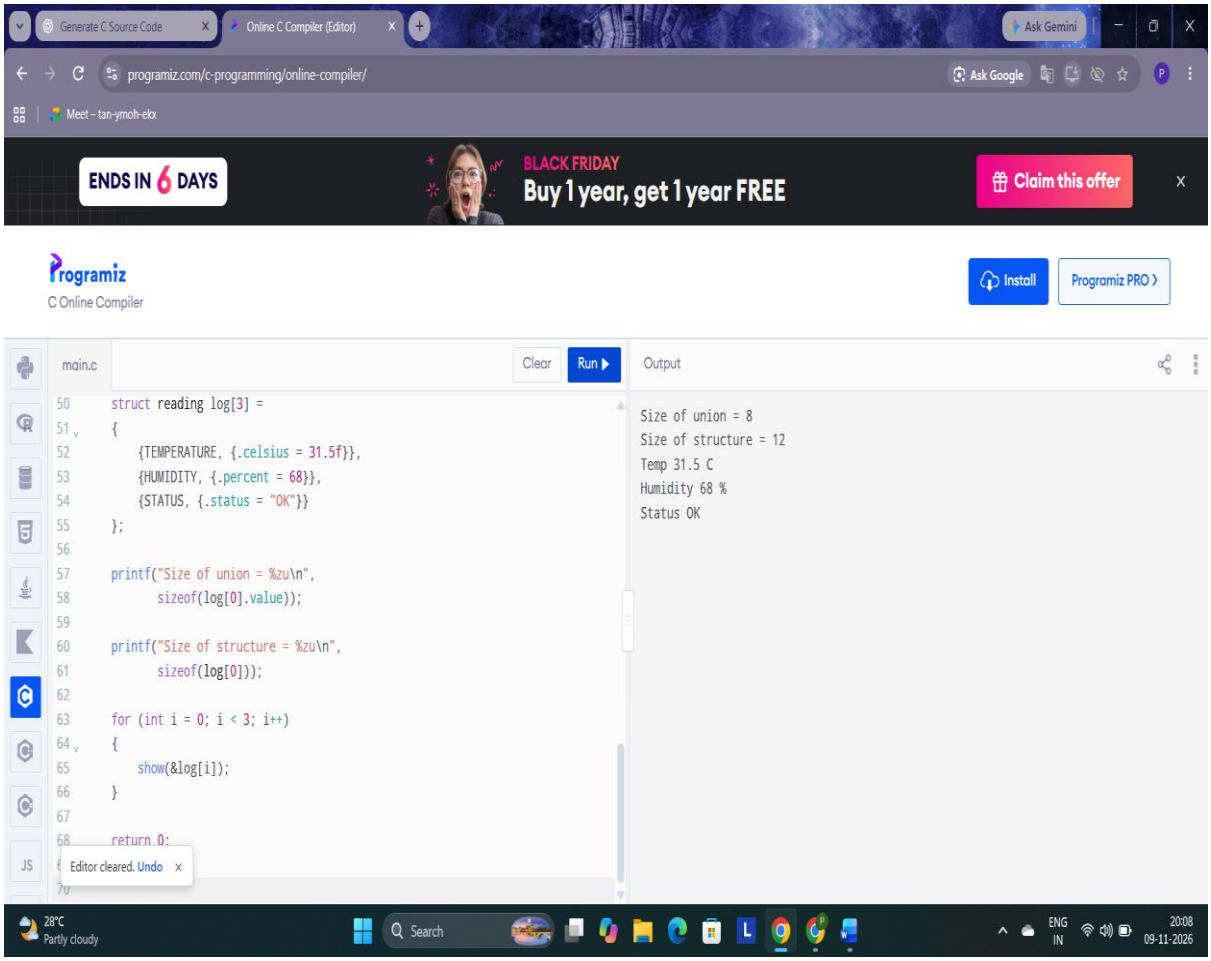
    printf("Size of union = %zu\n",
           sizeof(log[0].value));

    printf("Size of structure = %zu\n",
           sizeof(log[0]));

    for (int i = 0; i < 3; i++)
    {
        show(&log[i]);
    }

    return 0;
```

}



4)

#include <stdio.h>

struct account

{

int number;

float balance;

};

void addInterest(struct account *a, float rate)

```
{
    printf("Address inside function: %p\n",
           (void *)a);

    a->balance = a->balance +
                 a->balance * rate / 100;
}

int main()
{
    struct account list[3] =
    {
        {101, 5000},
        {102, 12000},
        {103, 800}
    };

    struct account *p = list;

    printf("Address in main: %p\n",
           (void *)p);

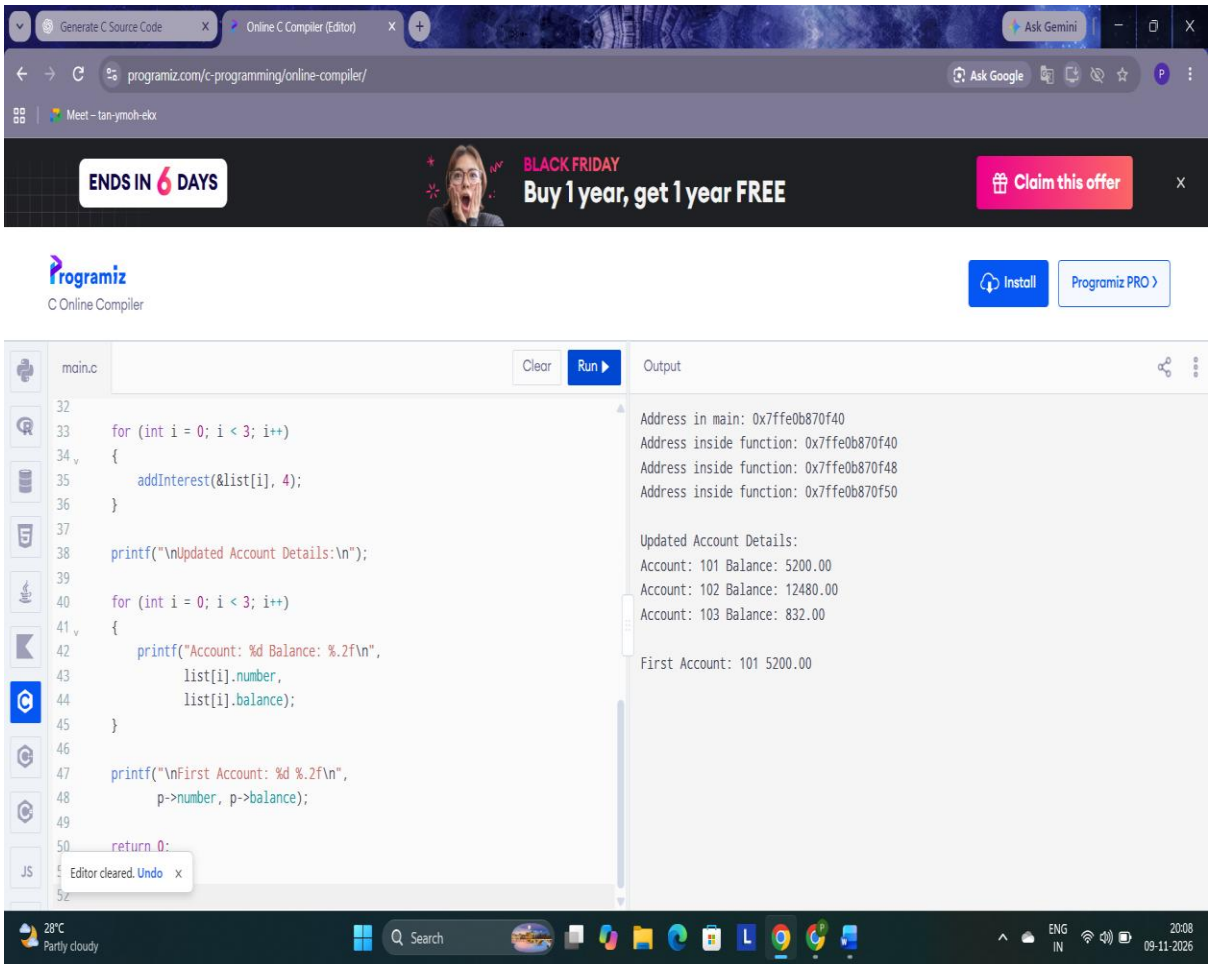
    for (int i = 0; i < 3; i++)
    {
        addInterest(&list[i], 4);
    }

    printf("\nUpdated Account Details:\n");
```

```
    for (int i = 0; i < 3; i++)
    {
        printf("Account: %d Balance: %.2f\n",
               list[i].number,
               list[i].balance);
    }

    printf("\nFirst Account: %d %.2f\n",
           p->number, p->balance);

    return 0;
}
```



6)

```
#include <stdio.h>
#include <string.h>

#define MAX 20
#define PARTS 3

struct Date
{
   int day, month, year;
};

enum ItemType
{
   PACKAGED,
   COOKED,
   COMBO
};

struct Item
{
   int id;
   char name[30];

   float price;
   int stock;
   float revenue;
```

```
    enum ItemType type;

    union
    {
        struct
        {
            struct Date expiry;
        } packaged;

        struct
        {
            struct Date prepared;
            char time[10];
        } cooked;

        struct
        {
            int count;
            int itemId[PARTS];
            int quantity[PARTS];
        } combo;

    } details;
};

int find_item(struct Item items[], int n, int id)
{
```

```
    for (int i = 0; i < n; i++)
    {
        if (items[i].id == id)
            return i;
    }

    return -1;
}

long date_number(struct Date d)
{
    int y = d.year;
    int m = d.month;

    if (m <= 2)
        y--;

    long days = 365L * y + y / 4 - y / 100 + y / 400;

    int month_days[] =
    {
        0, 31, 59, 90, 120, 151,
        181, 212, 243, 273, 304, 334
    };

    days += month_days[m - 1] + d.day;

    if (d.month > 2 &&
```

```
      (d.year % 400 == 0 ||
      (d.year % 4 == 0 && d.year % 100 != 0)))
   {
      days++;
   }

   return days;
}

int check_stock(struct Item items[], int n,
            int index, int quantity)
{
   if (items[index].type != COMBO)
      return items[index].stock >= quantity;

   for (int i = 0;
       i < items[index].details.combo.count; i++)
   {
      int id = items[index].details.combo.itemId[i];

      int required =
         items[index].details.combo.quantity[i] * quantity;

      int pos = find_item(items, n, id);

      if (pos == -1 || items[pos].stock < required)
         return 0;
   }
```

```
    return 1;
}

void record_sale(struct Item items[], int n,
                 int id, int quantity)
{
    int pos = find_item(items, n, id);

    if (pos == -1)
    {
        printf("Item not found!\n");
        return;
    }

    if (quantity <= 0)
    {
        printf("Invalid quantity!\n");
        return;
    }

    if (!check_stock(items, n, pos, quantity))
    {
        printf("Insufficient stock!\n");
        return;
    }

    if (items[pos].type == COMBO)
```

```
{
   for (int i = 0;
       i < items[pos].details.combo.count; i++)
   {
      int itemId =
         items[pos].details.combo.itemId[i];

      int required =
         items[pos].details.combo.quantity[i] * quantity;

      int component = find_item(items, n, itemId);

      items[component].stock -= required;
   }
}
else
{
   items[pos].stock -= quantity;
}

float amount = items[pos].price * quantity;

items[pos].revenue += amount;

printf("Sale successful!\n");
printf("Item: %s\n", items[pos].name);
printf("Quantity: %d\n", quantity);
printf("Bill Amount: Rs. %.2f\n", amount);
```

```
}

void expiry_warning(struct Item items[], int n,
                    struct Date today)
{
   printf("\nExpiry Warnings:\n");

   long current = date_number(today);

   int found = 0;

   for (int i = 0; i < n; i++)
   {
      if (items[i].type == PACKAGED)
      {
         long expiry =
            date_number(items[i].details.packaged.expiry);

         long difference = expiry - current;

         if (difference >= 0 && difference <= 2)
         {
            printf("%s expires within %ld day(s)\n",
                   items[i].name, difference);

            found = 1;
         }
      }
```

```
    }

    if (!found)
        printf("No items expiring within two days.\n");
}

void sort_revenue(struct Item items[], int n)
{
    for (int i = 0; i < n - 1; i++)
    {
        int max = i;

        for (int j = i + 1; j < n; j++)
        {
            if (items[j].revenue > items[max].revenue)
                max = j;
        }

        struct Item temp = items[i];
        items[i] = items[max];
        items[max] = temp;
    }
}

void billing_summary(struct Item items[], int n)
{
    sort_revenue(items, n);
```

```
    printf("\nDAILY BILL SUMMARY\n");

    printf("\nItem\t\tRevenue\t\tStock\n");

    float total = 0;

    for (int i = 0; i < n; i++)
    {
        printf("%-15s Rs. %-10.2f %d\n",
               items[i].name,
               items[i].revenue,
               items[i].stock);

        total += items[i].revenue;
    }

    printf("\nTotal Revenue = Rs. %.2f\n", total);
}

int main()
{
    struct Item items[MAX];

    int n = 4;

    struct Date today = {9, 11, 2026};

    // Packaged item 1
```

```
items[0].id = 1;
strcpy(items[0].name, "Biscuits");
items[0].price = 20;
items[0].stock = 10;
items[0].revenue = 0;
items[0].type = PACKAGED;
items[0].details.packaged.expiry =
   (struct Date){11, 11, 2026};

// Packaged item 2
items[1].id = 2;
strcpy(items[1].name, "Juice");
items[1].price = 30;
items[1].stock = 8;
items[1].revenue = 0;
items[1].type = PACKAGED;
items[1].details.packaged.expiry =
   (struct Date){10, 11, 2026};

// Cooked item
items[2].id = 3;
strcpy(items[2].name, "Samosa");
items[2].price = 15;
items[2].stock = 20;
items[2].revenue = 0;
items[2].type = COOKED;
items[2].details.cooked.prepared =
   (struct Date){9, 11, 2026};
```

```
strcpy(items[2].details.cooked.time, "10:30");

// Combo meal
items[3].id = 4;
strcpy(items[3].name, "Snack Combo");
items[3].price = 50;
items[3].stock = 0;
items[3].revenue = 0;
items[3].type = COMBO;

items[3].details.combo.count = 2;

items[3].details.combo.itemId[0] = 1;
items[3].details.combo.quantity[0] = 1;

items[3].details.combo.itemId[1] = 3;
items[3].details.combo.quantity[1] = 2;

printf("COLLEGE CANTEEN INVENTORY\n");

printf("\nAvailable Items:\n");

for (int i = 0; i < n; i++)
{
   printf("%d. %s - Rs. %.2f\n",
        items[i].id,
        items[i].name,
```

```
            items[i].price);
    }

    expiry_warning(items, n, today);

    printf("\nEnter item ID (0 to stop): ");

    int id, quantity;

    while (scanf("%d", &id) == 1 && id != 0)
    {
        printf("Enter quantity: ");

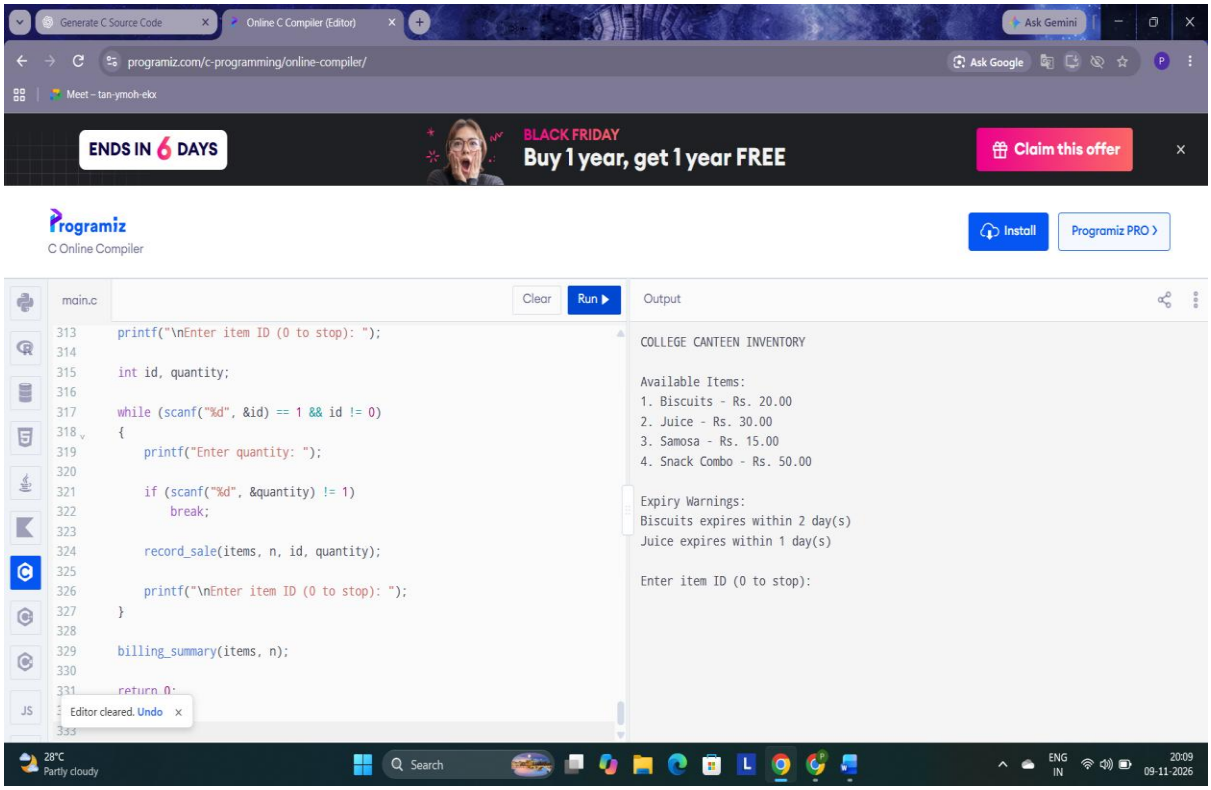
        if (scanf("%d", &quantity) != 1)
            break;

        record_sale(items, n, id, quantity);

        printf("\nEnter item ID (0 to stop): ");
    }

    billing_summary(items, n);

    return 0;
}
```


Generate C Source Code
Online C Compiler (Editor)
Ask Gemini
programiz.com/c-programming/online-compiler/
Ask Google
Meet – tan-ymoh-ekx
ENDS IN 6 DAYS
BLACK FRIDAY
Buy 1 year, get 1 year FREE
Claim this offer
Programiz
C Online Compiler
Install
Programiz PRO
main.c
Clear
Run
Output
313 printf("\nEnter item ID (0 to stop): ");
314
315 int id, quantity;
316
317 while (scanf("%d", &id) == 1 && id != 0)
318 {
319 printf("Enter quantity: ");
320
321 if (scanf("%d", &quantity) != 1)
322 break;
323
324 record_sale(items, n, id, quantity);
325
326 printf("\nEnter item ID (0 to stop): ");
327 }
328
329 billing_summary(items, n);
330
331 return 0;
Editor cleared. Undo
COLLEGE CANTEEN INVENTORY
Available Items:
1. Biscuits - Rs. 20.00
2. Juice - Rs. 30.00
3. Samosa - Rs. 15.00
4. Snack Combo - Rs. 50.00
Expiry Warnings:
Biscuits expires within 2 day(s)
Juice expires within 1 day(s)
Enter item ID (0 to stop):
28°C
Partly cloudy
Search
ENG
IN
20:09
09-11-2026